Wii™ Sports FPGA

Final Project Demo

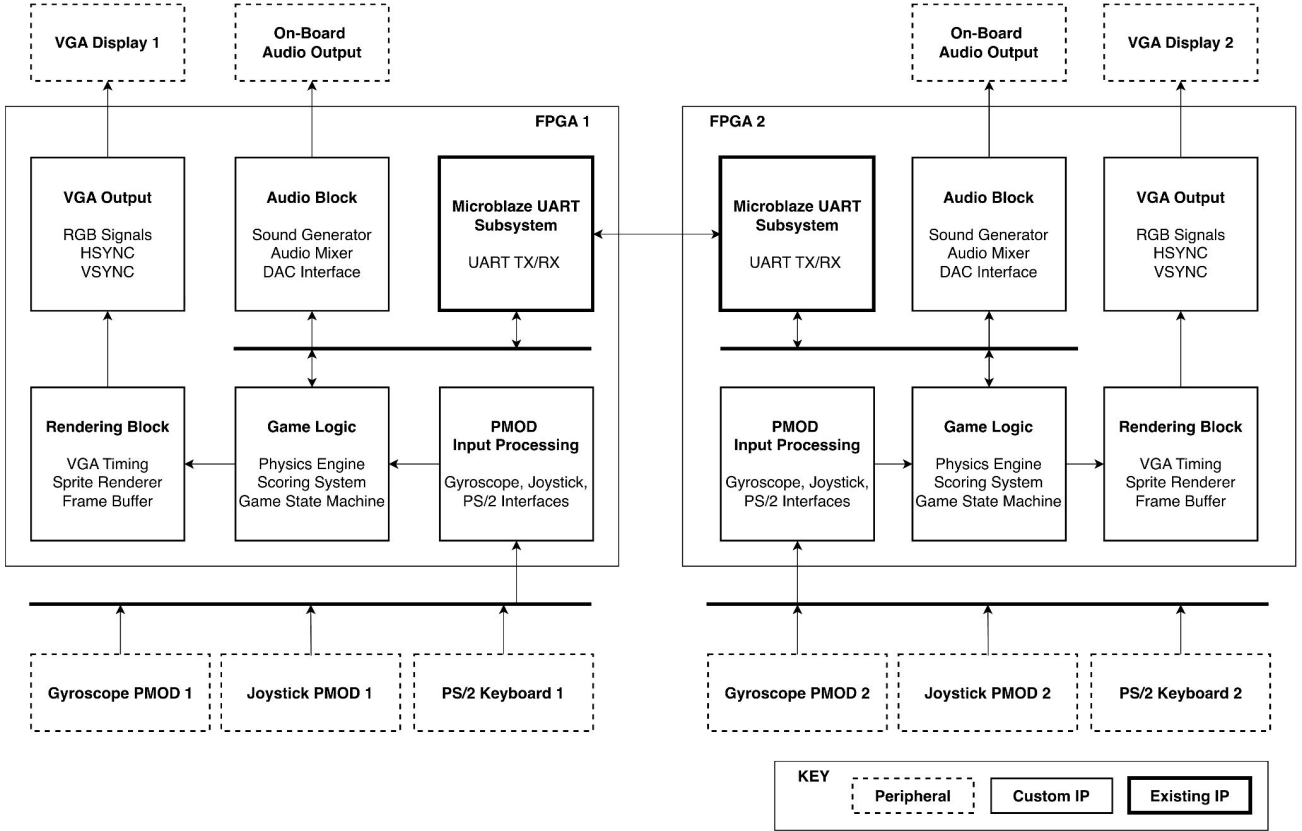
Team (#7): Celine K, Hannah L, Joyce C. Uzma F.

Initial Project Overview

- 2-player Wii Sports Resort archery game
  - Interface 2 FPGAs for 2-player game
  - Custom IPs for VGA reader and writer
  - PS/2 and gyroscope PMODs

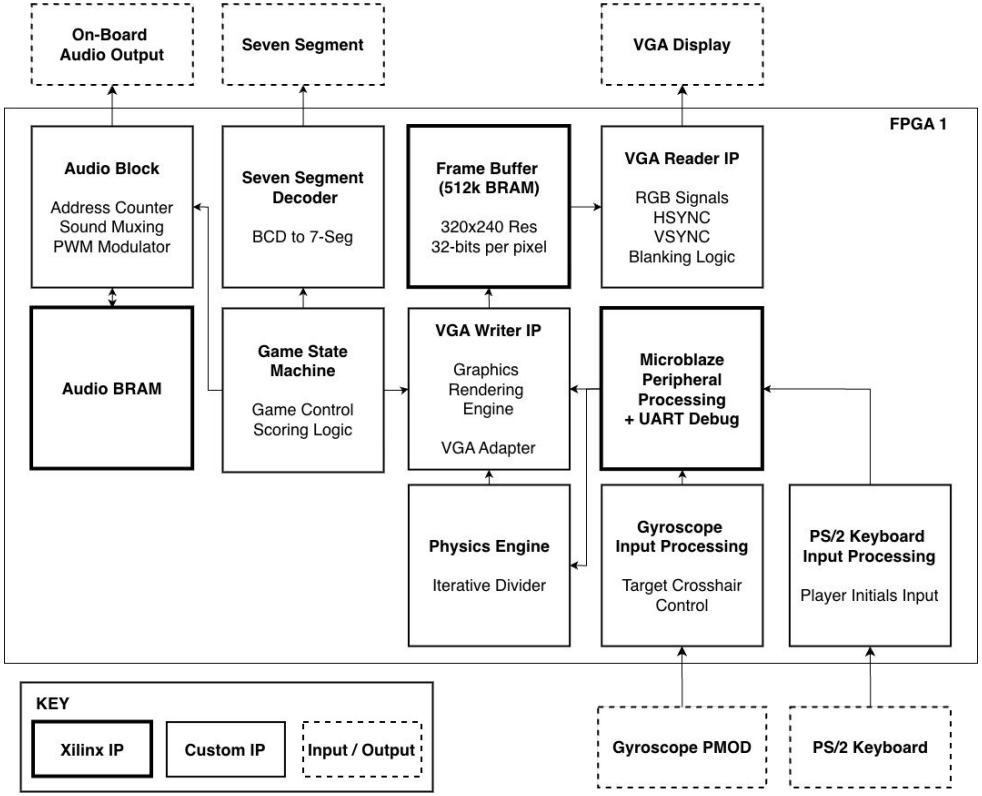
Fig 1: Original Wii Sports Resort archery game [1].

Initial Goals & Block Diagram



Final Implemented Project

- **Significant Revisions**
  - Removed joystick
  - Removed dual-FPGA
- VGA output (without prebuilt IPs) was more **complex** than anticipated
  - Pipelined graphics rendering engine
  - Writing 76,800 RGB pixels to BRAM and reading to drive VGA
- Focus on **physics IP**
- Seven Segment Additions
  - Score Display



Dual-FPGA vs. Physics IP Tradeoff

➔ Time constraints led us to making a decision between a custom physics IP and a dual-FPGA game

- **Dual-FPGA game:** Full duplex (bidirectional) UART using Microblaze with a custom packet structure
  - 0.25* complexity (focus on protocol design, system integration)

- **Physics IP:** Custom IP-Core with arithmetic and pipelining
  - 1.25* complexity (focus on **algorithmic hardware design**)

Complexity Score

Component	Score
VGA Custom Reading IP (IP Core implemented in FPGA Hardware)	0.25*
VGA Custom Reading IP (VGA output without MicroBlaze involvement)	1.0*
Graphics Rendering Engine (IP Core implemented in FPGA Hardware)	0.75*
Physics Engine	1.25*
On-board Audio Output Port	0.5
Gyroscope PMOD	0.75
PS/2 Mouse/Keyboard Connector PMOD	0.5
7-Segment Display	0.20
Final Score	5.2*

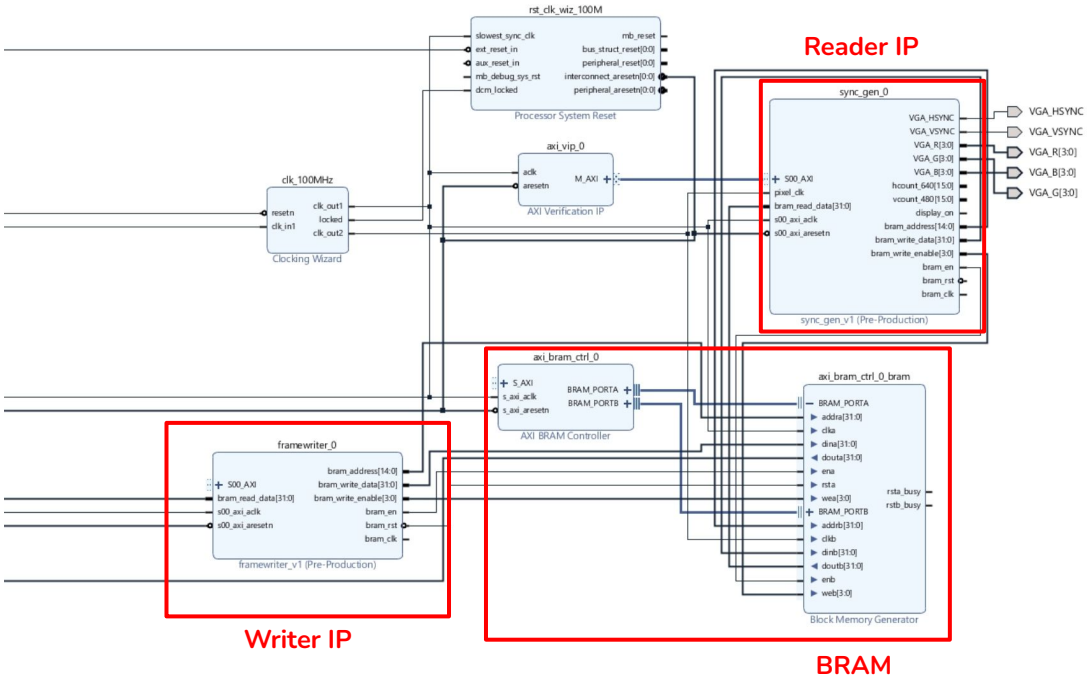
Custom VGA Adapter IP Overview

Writer IP

- ➔ Interfaces with game state machine, physics engine and peripheral inputs
- ➔ Computes and writes information for every pixel to VGA BRAM

Reader IP

- ➔ Takes BRAM and writes to VGA screen
  - ◆ Outputs VSYNC, HSYNC, and RGB



Custom VGA Adapter IP Features

- **Graphics Rendering Engine (0.75 pts)**
  - **3-stage pipelined** architecture with state machine coordination → reduced TNS by **20x**
  - Geometry calculations for target rings, text boxes, and crosshair
  - Internal font ROM with 40+ ASCII characters and bitmap extraction
  - Six distinct game screen modes with priority-based pixel multiplexing
  - Text scaling engine to create three different text sizes (1x, 2x, 4x)
  - Dynamic character overlay from external bitmaps

- **VGA Writer + Reader IP (1 pt + 0.25 pts)**
  - Frame sync as per VGA protocol & coordinate management across 76.8k pixels
  - BRAM interface with address generation and write enable control

Effects of Pipelining

- Implemented a 3-stage pipeline:
  - **READ** – ASCII character selection and geometry calculations per pixel
  - **WAIT** – ROM lookup, distance squared computation, bitmap extraction
  - **WRITE** – Final pixel color muxing with priority layering

Timing

Worst Negative Slack (WNS):	-13.895 ns
Total Negative Slack (TNS):	-892.361 ns
Number of Failing Endpoints:	715
Total Number of Endpoints:	29638

Implemented Timing Report

Design Timing Summary

Setup

Worst Negative Slack (WNS):	-6.013 ns
Total Negative Slack (TNS):	-24.542 ns
Number of Failing Endpoints:	9
Total Number of Endpoints:	29087

Hold

Worst Hold Slack (WHS):	0.010 ns
Total Hold Slack (THS):	0.000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	29071

Pulse Width

Worst Pulse Width Slack (WPWS):	3.000 ns
Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	14227

Timing constraints are not met.

Pipelining reduced negative slack by a factor of 10

Physics Engine Overview

- Implements wind physics affecting arrow landing position.
  - Constant wind speed vector based on difficulty level
  - Calculate arrow landing position based on aim position and wind speed.

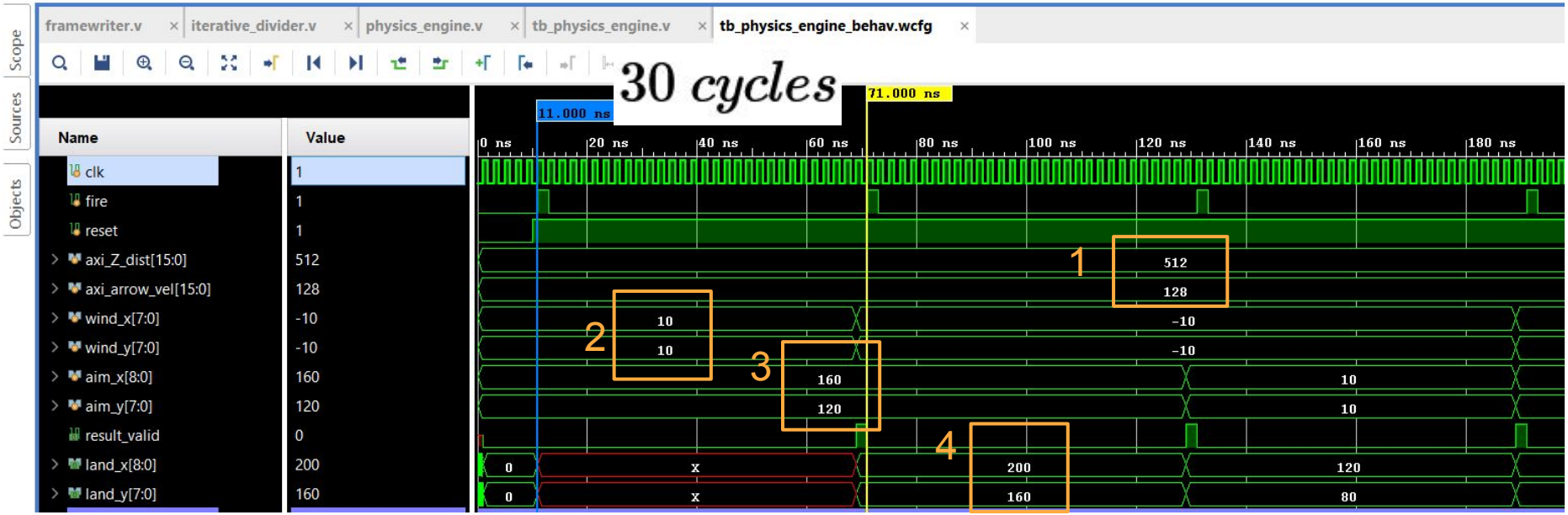
Physics Engine Features

- Use Q8.8 fixed point arithmetic using signed magnitude
- 4-stage register pipeline for arithmetic operations
- Custom divider IP module to avoid combinational logic delays
    - minimized resource usage by avoiding use of DSP slices

Constant Velocity Drift Model

Stage	Operations	
1	Compute time of flight – how long the arrow is in the air	$t = Z_{dist} / V_{arrow}$
2	Compute delta_x = wind_x_mag * (Z_dist/arrow_vel)	$\Delta X = V_{wind_x} \times t$
3	Compute delta_y = wind_y_mag * (Z_dist/arrow_vel)	$\Delta Y = V_{wind_y} \times t$
4	Compute land_x and land_y, extract integer value and clamp w/in VGA frame dimensions	$Land_X = Aim_X \pm \Delta X, Land_Y = Aim_Y \pm \Delta Y$

Constant Velocity Drift Model



1 $t = \frac{512}{128} = 4.0$

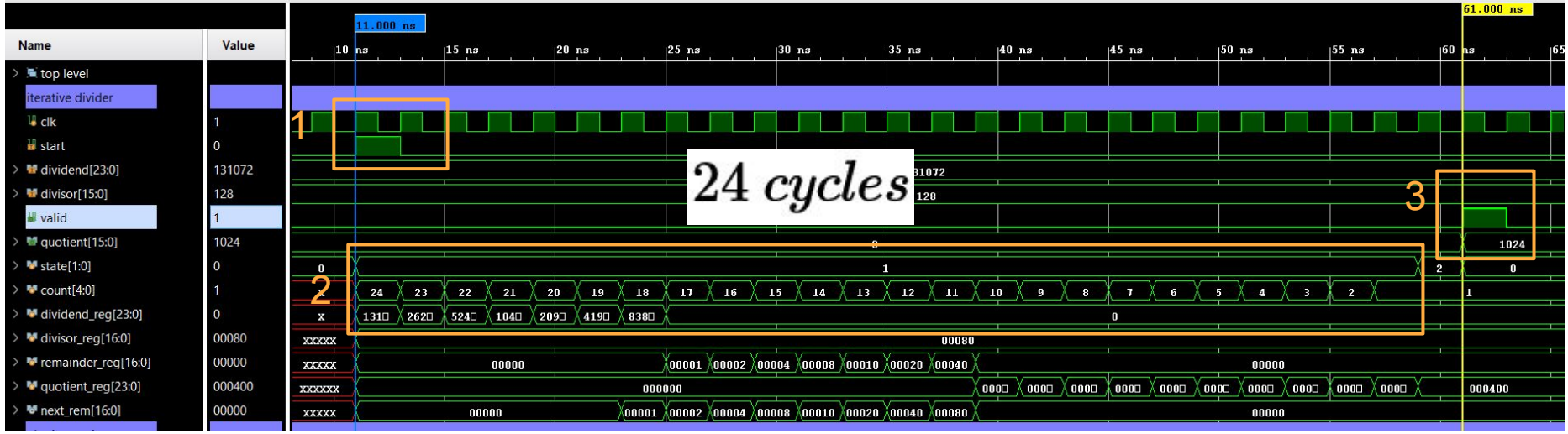
2 & 3 $\Delta x = 10 * 4 = 40,$ $\Delta y = 10 * 4 = 40$

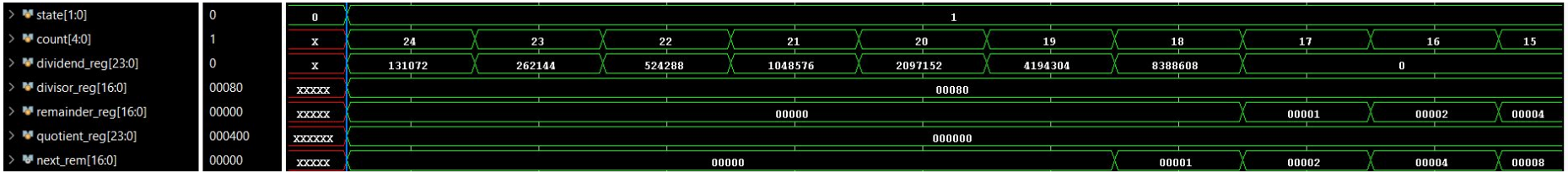
4 $land_x = 160 + 40 = 200$ $land_y = 120 + 40 = 200$

Iterative Divider

Cycle	Operations	FSM State
0	The start (fire) signal is sampled high.	ST_IDLE
1	Initialization: Registers are loaded, and count is set to 24.	ST_IDLE → ST_DIV
2-25	2-24: shifts and subtracts 25: count reaches 1 and the state transitions to `ST_DONE`	ST_DIV
26	The final result is at the `quotient` output, and `valid` is high.	ST_DONE
27	The module returns to idle, and valid returns to low.	ST_IDLE

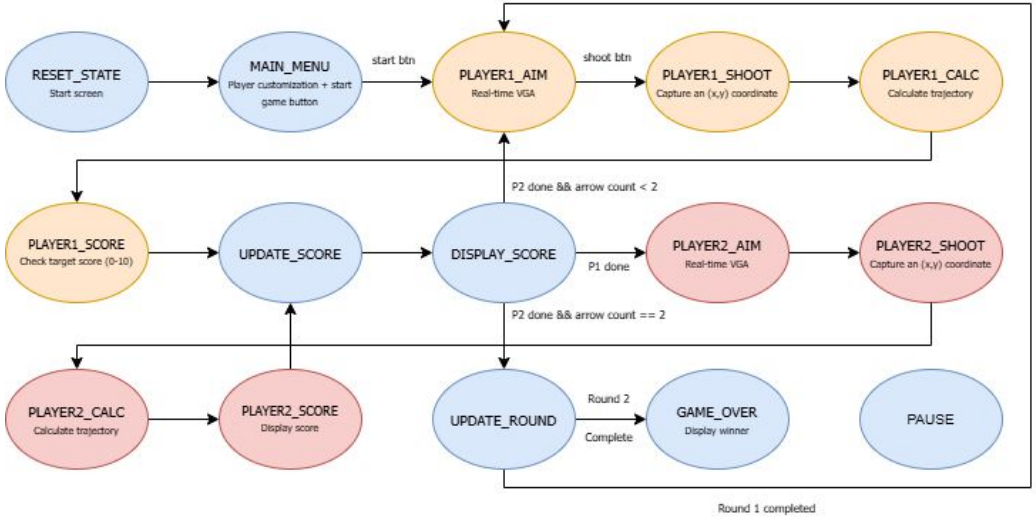
Iterative Divider





Game FSM

- Controls game flow through states such as MAIN_MENU, PLAYER1_SHOOT, PLAYER1_SCORE, DISPLAY_SCORE, etc.
- State changes often depend on button presses (one-cycle pulse) or timers
- Difficulty settings (easy, medium, hard) that affect windspeed
- Tracks gameplay data and packs it as output for the framewriter
    - // [24:23] = difficulty
    - // [22:16] = p1_score
    - // [15:9] = p2_score
    - // [8:6] = current_round
    - // [5:4] = arrow_count
    - // [3:0] = current_state



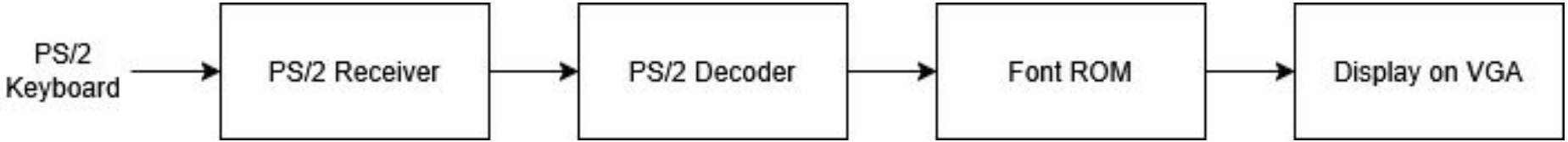
Gyroscope Pmod

- Calculate player aim coordinates based on angular velocity data
  - Integrates velocity data to calculate distance and then maps to VGA coordinate
  - Reduce noise and drift

Feature	Description
Calibration	Determine bias value of gyroscope at rest by averaging 512 samples for x, y, z axis data.
Noise Filtering	Deadzone of 75 to ignore small movement values and sensor noise to prevent drift and produce reasonable aim sensitivity.
Dynamic Bias Tracking	Continually update bias values if gyroscope is at rest to mitigate drift from sensor noise and variation.
Velocity Integration	Accumulators for angular velocity data calculate position.

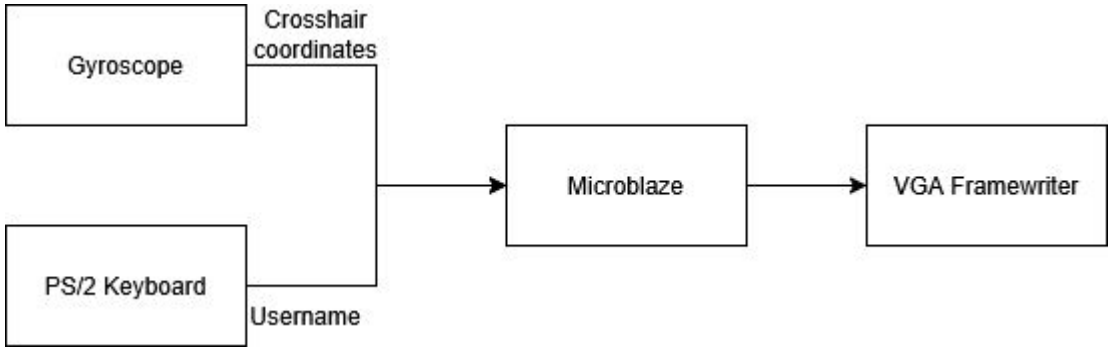
PS/2

- ➔ Player name input
- ➔ Receives and decodes alphabetical and numerical input



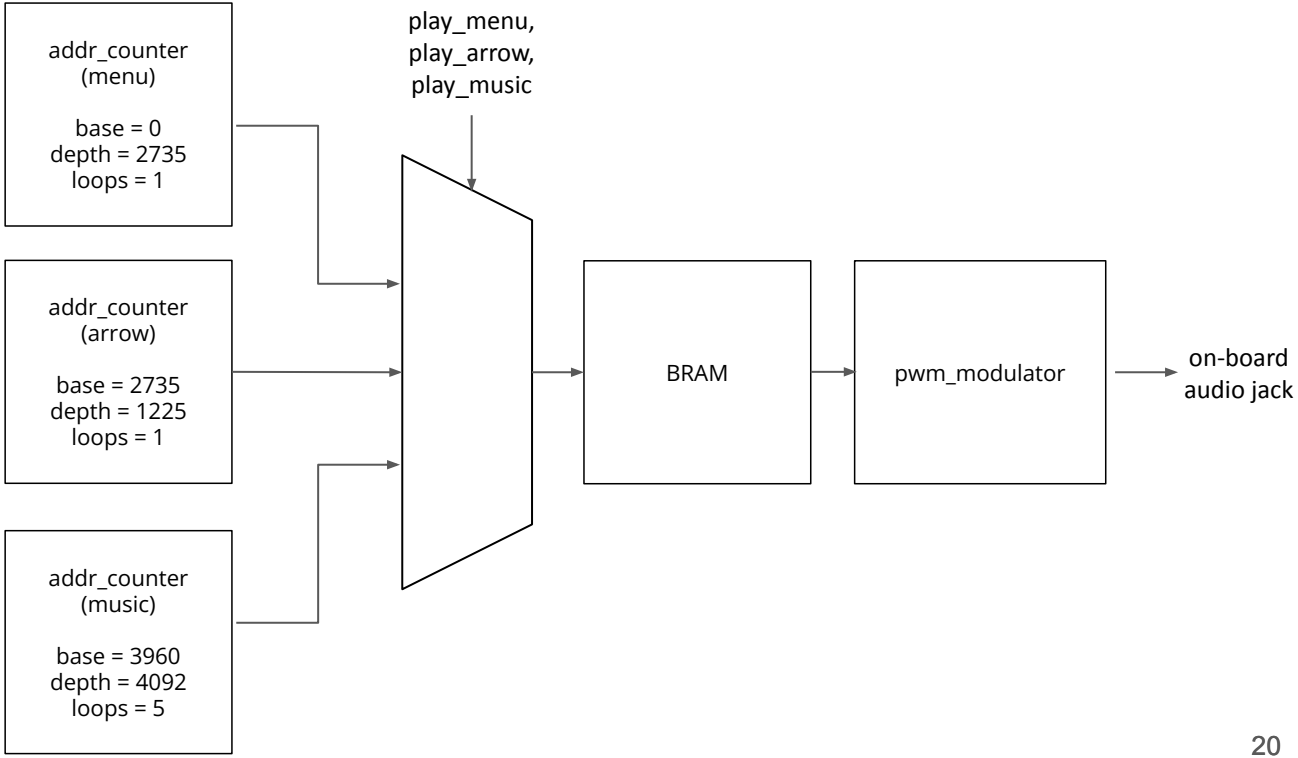
Software Flow

➔ Event loop running on Microblaze processor polls peripheral data and passes to VGA subsystem for display



Onboard Audio

- Used sfxr.me to generate 8-bit audio sounds for 3 game events
  - For 'music' generated 4 tones and mixed them together
  - Wrote python script to convert sounds from raw hex to a .coe file



Potential Improvements

If given more time, we would:

- ➔ Improve graphics
  - ◆ Continuous statistics for physics engine (e.g. wind speed, difficulty, original and new coordinates)
  - ◆ Animations
- ➔ Add dual-FPGA functionality
- ➔ Variable physics engine Z-distance & arrow velocity inputs

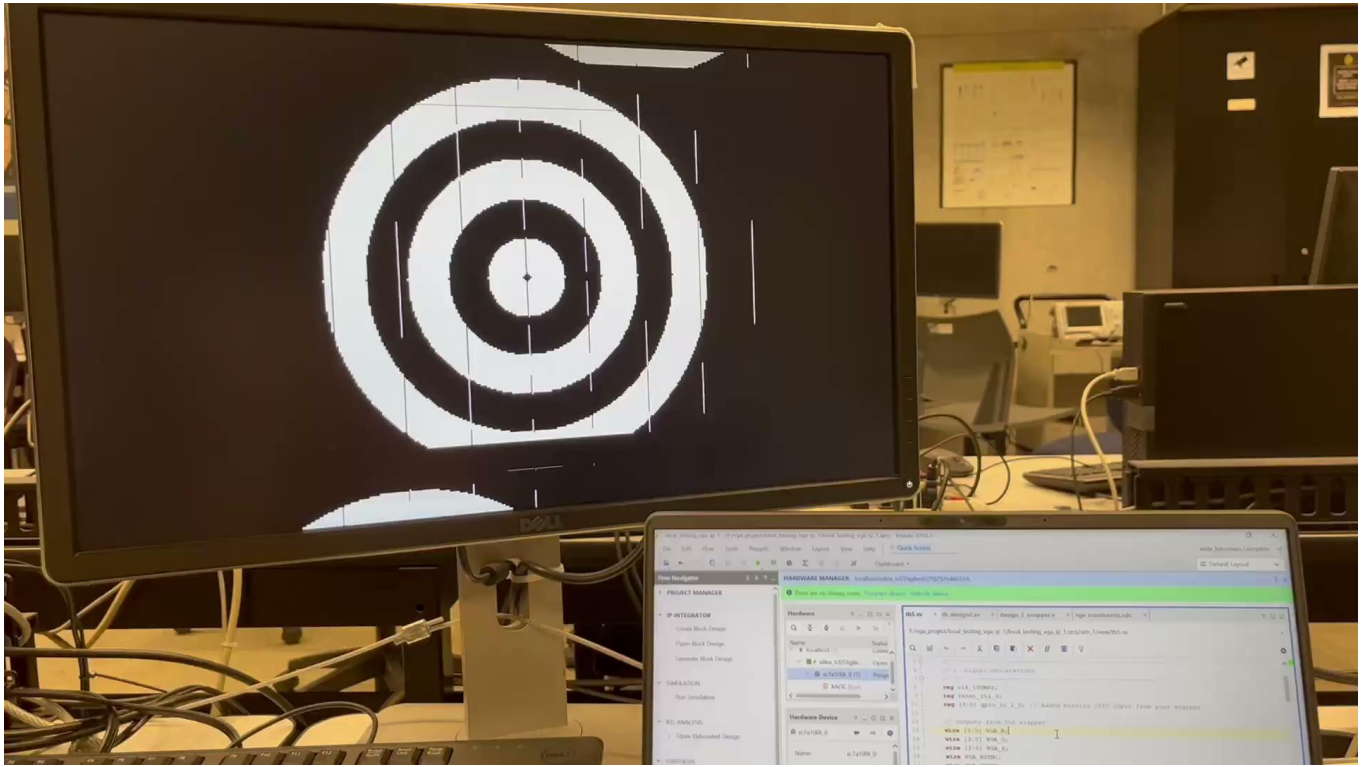
Questions?

References

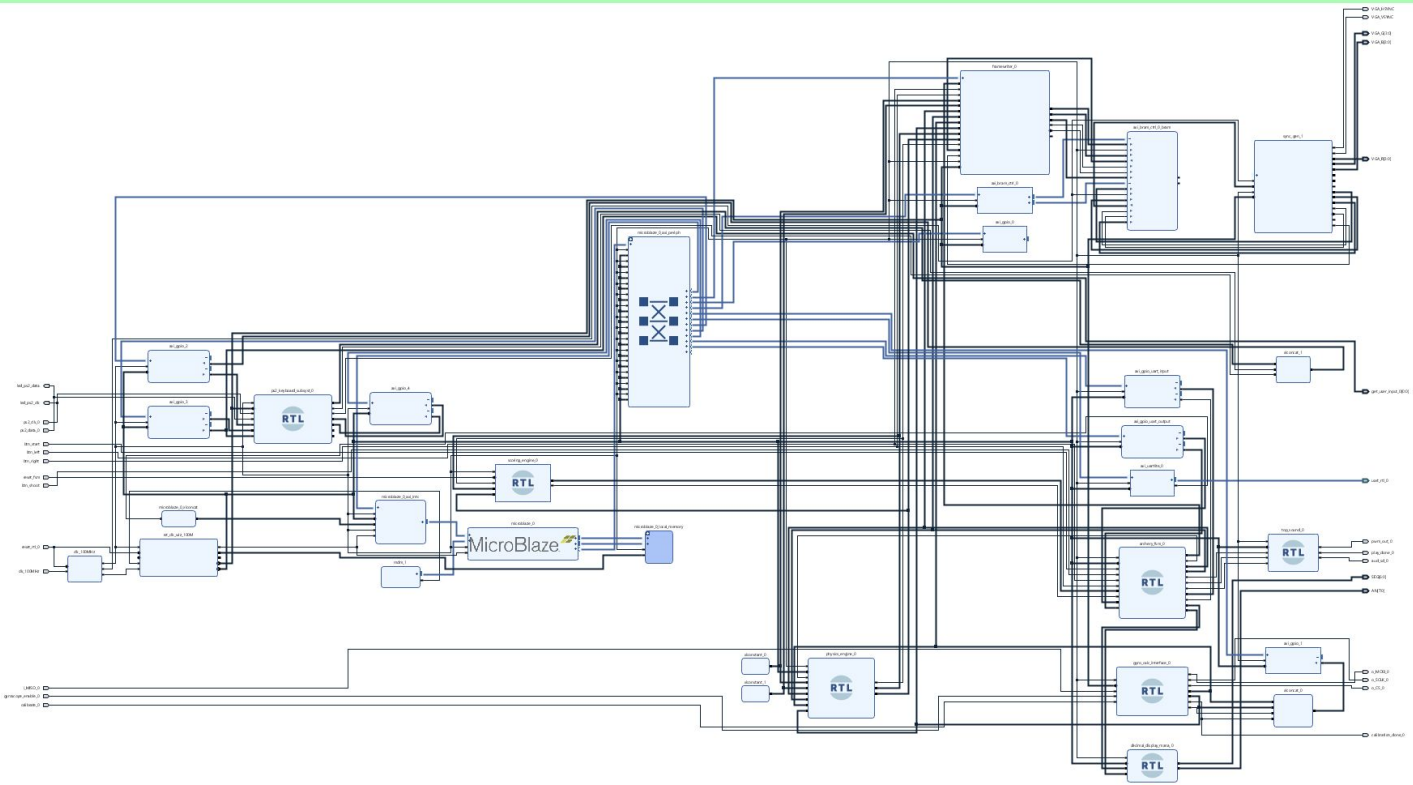
[1] Wii Sports Wiki. "Archery." Wii Sports Wiki. Accessed: Jan. 30, 2025. [Online]. Available: https://wiisports.fandom.com/wiki/Archery

Appendices

VGA IP Demo Target



Vivado Block Diagram



Main Components

- ➔ VGA Custom IPs
- ➔ Gyroscope PMOD
- ➔ PS/2 PMOD
- ➔ Physics + Scoring Engines
- ➔ detailed explanation of the compute/logic complexity of any custom hardware blocks
- ➔ explanation of software flow in processor system (interrupts, event loop, etc.)
- ➔ explanation of bus configuration, dataflow schema, and data storage (if applicable)

VGA Writer IP (0.75 pts)

Brute force	Our implementation	Optimization
one giant `always @(*)` block with comb. logic for: • Distance from centre calculation • ROM lookup for ASCII characters • Decide pixel color	3-stage implementation: 1. Calculates stg1_dx, stg1_dy and selects stg1_ascii. 2. Calculates stg2_dist_sq = **dx^2** + **dy^2**. Fetches rom_data. Dedicated cycle for multiplication using **DSP slice**. 3. Decides the final hex color (RED, BLUE, etc.) for the BRAM	pipeline stages → 10x reduction in **Total Negative Slack** (TNS)
use LUTs and FFs for multiplication	used DSP slice for target band calc	less resource usage

Resource Usage & Timing Stats

DRC Violations

Summary: 25 warnings

Implemented DRC Report

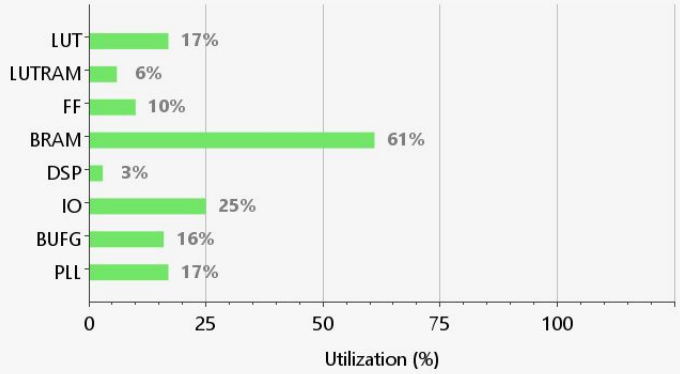
Timing — **Setup** | Hold | Pulse Width

Worst Negative Slack (WNS):	-6.293 ns
Total Negative Slack (TNS):	-38.642 ns
Number of Failing Endpoints:	56
Total Number of Endpoints:	29077

Implemented Timing Report

Utilization — Post-Synthesis | **Post-Implementation**

Graph | Table



Power — **Summary** | On-Chip

Total On-Chip Power:	**0.33 W**
Junction Temperature:	**26.5 °C**
Thermal Margin:	58.5 °C (12.7 W)
Effective ϑJA:	4.6 °C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Implemented Power Report

Iterative Divider

Brute force	Our implementation	Optimization

significant **LUT and FFs** used
`dividend_reg / divisor_reg`
	reuses a **single subtractor** and **shift register**
	`if (next_rem ≥ divisor_reg)`
	`remainder_reg ≤ next_rem - divisor_reg;`
		less resource usage

0-1 stages → **long critical path**	24-stage iterative division + 4-stage pipeline for physics calculation	pipeline stages, higher Fmax